

Fast API BASIC

Write By. 김지훈

Contents

- 01.** Web 의 개념과 Fast API
- 02.** Request 와 Parameter
- 03.** 비동기통신과 AJAX
- 04.** Cookie 와 Session
- 05.** DB 접속 과 C.R.U.D.
- 06.** 게시판의 개념과 제작

Contents

07. File Upload & Download

08. 데이터 Crawling

09. Back ground 작업

강의 목표

웹의 기본개념 이해와 Fast API 를 활용한 서비스 구현

Web 과Fast API 기본

Web 과 Fast API 의 개념
을 배우고 개발 환경을
설치해 본다.

Fast API 와 DB 연동

Fast API 의 기본 기능과
DB 연동 방법을 배운다.

Fast API 를 활용한 서비스 제작

Fast API 를 통해 서비스를
작성해 본다.

01.

Web 의 개념과 Fast API

Web 의 개념

✓ Server 와 Client

- 아래의 예시를 보고 client와 server 에 대해 생각해 보자
- client 가 보내는 요청인 request 와 server 의 응답인 response 가 무엇인지도 확인해 보자



- client : 이 물건을 계산해 주세요 (request)
- Server : 500원 입니다. (response)
- Client : 1000원 드렸습니다. (request)
- Server : 500원 거슬러 드렸습니다. (response)

Web 의 개념

☑ Server 와 Client

- client 는 Front-End 라고 부르며 HTML, CSS, Java script 등으로 구현 가능
- Server 는 Back-End 라고 부르며 Fast API 로 구현 가능

CLIENT



HTML,CSS,java script

SERVER



Fast API

Fast API

Fast API 란?

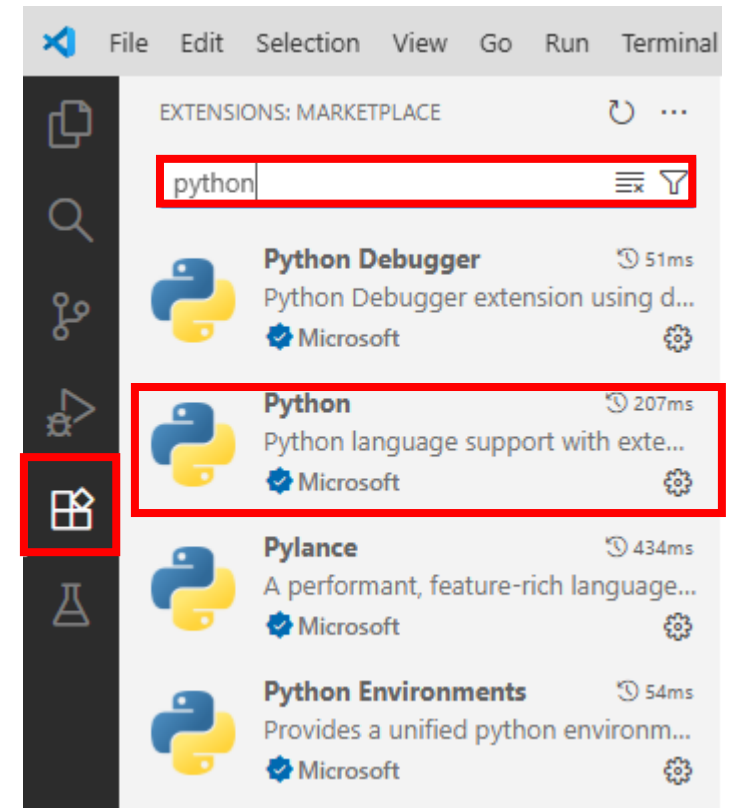
- Fast API 는 FLASK, Django 와 더불어 Python 의 대표 웹 프레임워크
- API 서버 개발에 최적화 되어 있음

항목	설명
빠른 성능	- Starlette와 uvicorn 기반으로 비동기 처리 시 성능 매우 우수
쉬운 개발	- python의 타입 힌트를 활용한 직관적인 코드 작성 가능 - Flask나 Django보다 더 적은 코드로 강력한 API 구축 가능
유지보수 우수	- 코드가 명확하고 테스트 작성이 쉬워 유지보수에 유리
자동 문서화	- 자동으로 Swagger UI, ReDoc 문서를 생성해 줘서 문서화가 간편

Fast API

✓ Fast API 설치(vs code 버전)

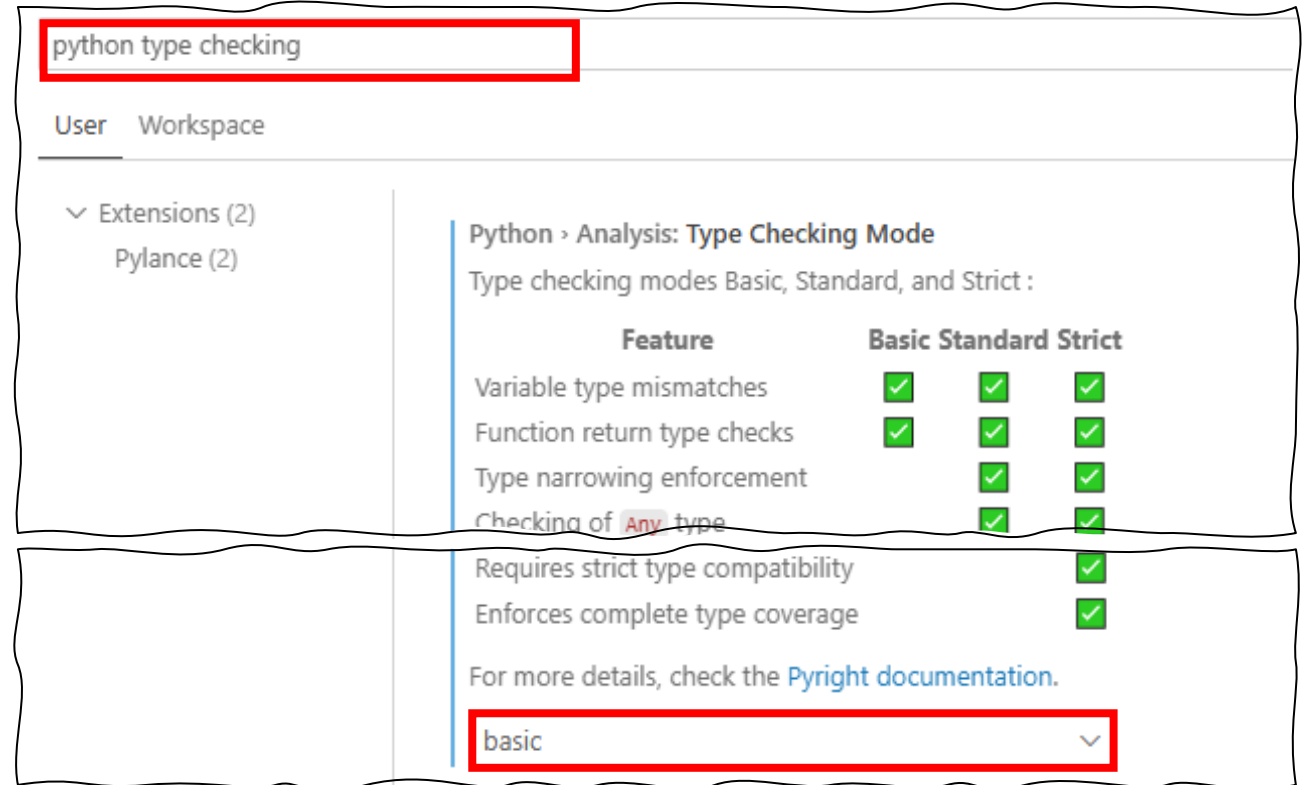
- 이미 Python 과 VS code 설치 되었으므로 VS CODE 에 Plugin 을 설치
- Python 만 설치해도 연관된 plugin 들도 설치됨



Fast API

✓ Fast API 설치(vs code 버전)

- Ctrl +,(Setting) 으로 이동 후 python type checking 입력
- 이후 basic 선택



Fast API

✓ Fast API 설치(vs code 버전)

- Ctrl + shift + p 이후 settings.json 입력 UserSettings 선택



- 아래 내용 추

```
{  가
  "editor.fontSize": 24,
  "liveServer.settings.donotShowInfoMsg": true,
  "workbench.colorTheme": "Default Light+",
  "python.analysis.typeCheckingMode": "basic",
  "python.analysis.autoImportCompletions": true,
  "editor.suggestSelection": "sprout", // 추천 설정 (가장 관련성 높은 항목을 미리 선택)
}
```

Fast API

✓ Fast API 설치(vs code 버전)

- Terminal > new terminal 선택(ctrl+') 하여 터미널 실행
- 기본 powershell 이므로 화살표 눌러 Command Prompt 로 변경



PROBLEMS OUTPUT DEBUG CONSOLE TERMINAL PORTS

PS D:\STUDY\vs_code\01_start> █



PROBLEMS OUTPUT DEBUG CONSOLE TERMINAL PORTS

Microsoft Windows [Version 10.0.19045.2006]
(c) Microsoft Corporation. All rights reserved.

D:\STUDY\vs_code\01_start> █

Fast API

✓ Fast API 설치(vs code 버전)

- python -m venv .env(가상환경 이름) 으로 가상환경 생성 후 실행

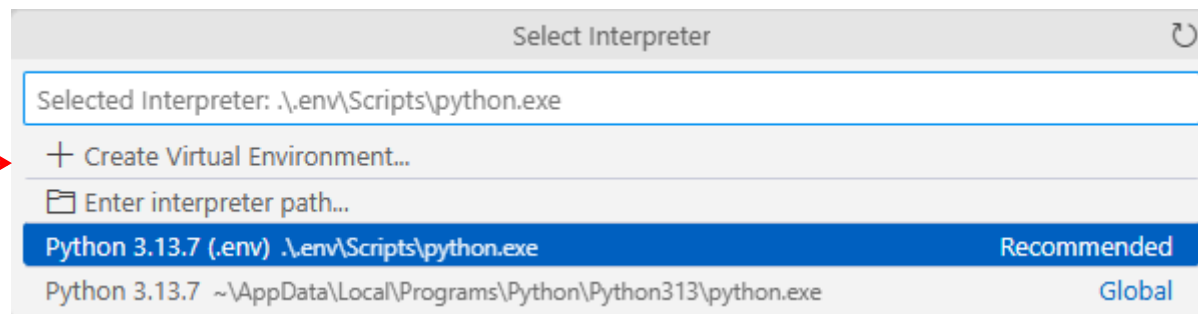
```
python -m venv .env  
.env\Scripts\activate.bat
```

- ctrl + shift + p > Python select interpreter 선택
- 이후 가상 환경 python 선택



- Fast api 와 uvicorn 설치

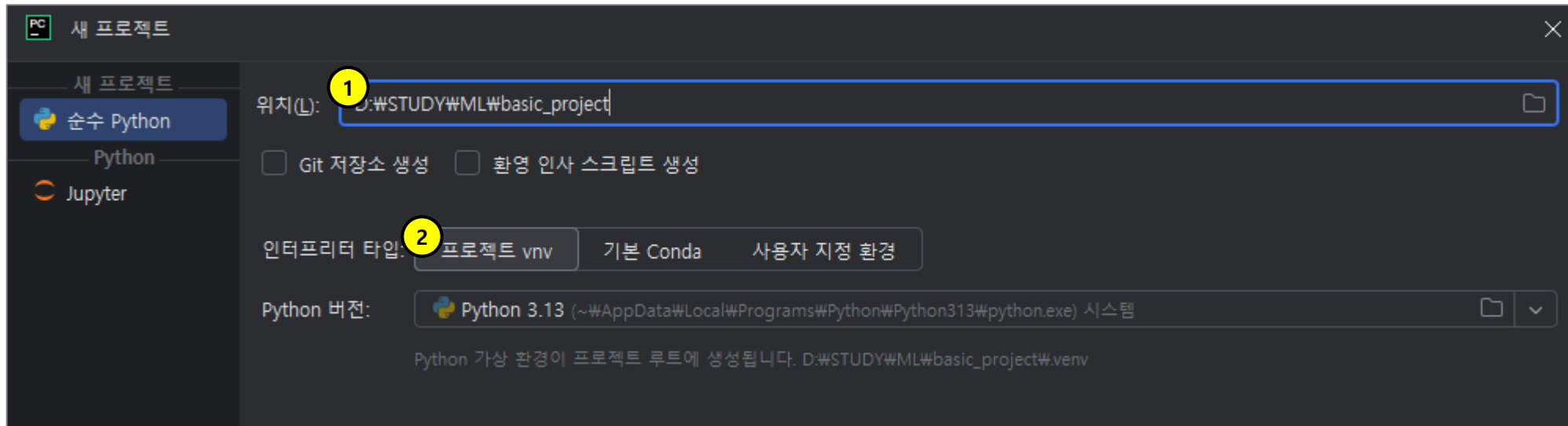
```
pip install fastapi uvicorn
```



Fast API

✓ Fast API 설치(PyCharm 버전)

- (1번) 프로젝트 위치와 이름을 지정해 주자
- (2번) 인터프리터 타입은 .VENV(가상환경) 프로젝트로 생성해 주자



Fast API

Fast API 설치(PyCharm 버전)

- (1번) 터미널 메뉴를 실행
- Fastapi 와 uvicorn 설치

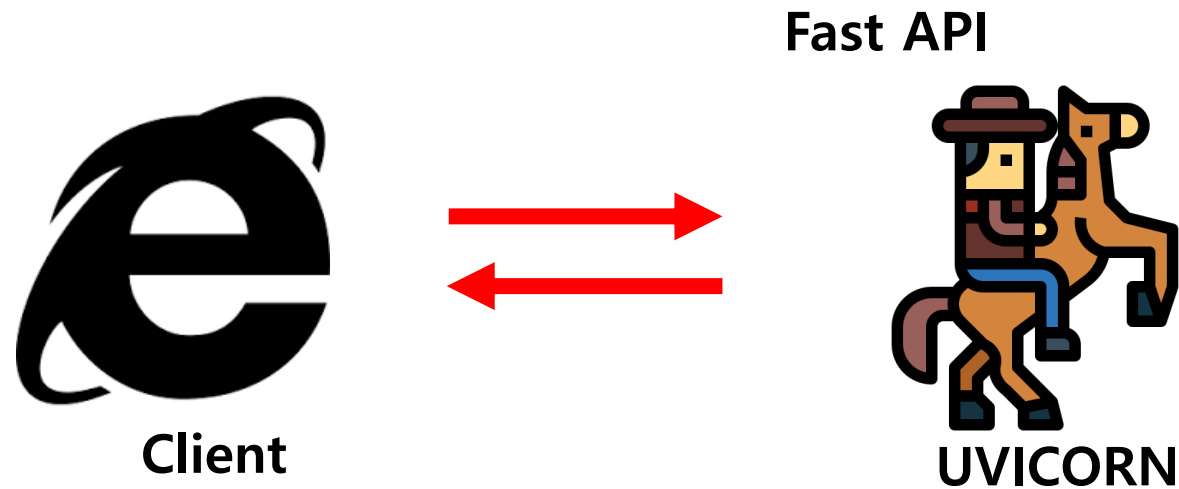


 `pip install fastapi uvicorn`

Fast API



- http 프로토 콜을 통해서 요청과 응답을 python 으로 구현하기엔 복잡하다.
- 그래서 간단하게 처리 할 수 있는 Uvicorn 같은 Web server 를 사용한다.
- 그리고 Uvicorn 의 환경에서 Fast API 가 실제적인 처리 로직을 수행 한다.



Fast API



Fast api 구동

- 아래 내용을 작성하여 서버를 실행해 보자

```
from fastapi import FastAPI

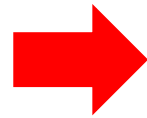
app = FastAPI()

@app.get("/")
def main():
    return {"message": "Fast API"}
```

실습파일 : 01_start/main.py

• 실행 결과

```
{"message": "Fast API"}
```



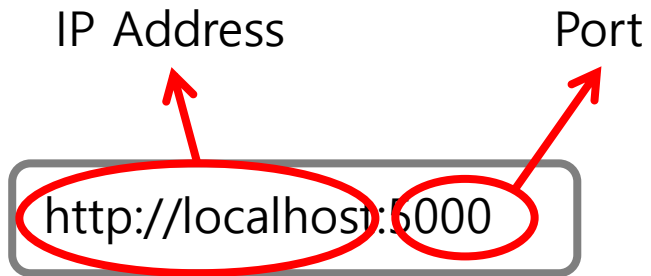
```
1 uvicorn main:app --reload
  # localhost 외 IP로 접근하고 싶다면?
2 --host=0.0.0.0
  # 포트를 변경하고 싶다면
3 --port=5000
```

1. main.py 의 app 변수의 객체를 실행하고 코드 변경 시 자동 서버 재시작
2. 어떠한 IP 라도 수락(기본 localhost 또는 127.0.0.1 만 가능)
3. 5000번 포트에서 시작(기본 8000번)

Fast API

☑ Fast api 구동

- 서버를 동작하면 실행하는 서버의 IP, Port 등을 주소창을 통해 알 수 있다.



- 결과 화면

```
{"message": "Fast API"}
```

Summary

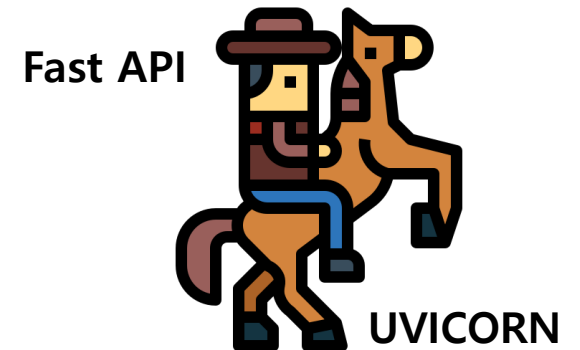
✓ Web

- Web 은 server 와 Client 가 존재함
- Client 가 보내는 요청인 Request, server 가 보내는 응답인 Response 가 존재함



✓ Fast API

- UVICORN : 웹 상의 요청과 간단하게 처리 할 수 있는 Web server
- Fast API : Uvicorn 환경에서 실제적인 처리 로직 처리를 수행하는 Frame work



02.

Request 와 Parameter

Request 와 Parameter

request

- Server 와 Client 간에는 요청(request)과 응답(response)이 오간다.
- Request 객체는 Client 로 부터 온 요청에 관한 모든 정보를 가지고 있다.
- 쇼핑몰에서 구매 요청 시 제공하는 정보를 생각 해 보자

• 구매 요청 시 제공 정보

상품명 선택 옵션
상품가격 구매자 이름
결제 수단 구매 수량
배송 주소

• 서버 요청 시 제공 정보

```
↑ @get(self, key)      Mapping
↓ @close(self)      Request
↑ @__sizeof__(self)      object
↑ len      len(expr)
↑ @app      HTTPConnection
↓ @keys(self)      Mapping
↓ @headers      HTTPConnection
↓ @cookies      HTTPConnection
@auth      HTTPConnection
@base_url      HTTPConnection
@body(self)      Request
client      HTTPConnection
```

Enter(을) 눌러 삽입하거나 Tab(을) 눌러 바꿉니다.

Request 와 Parameter

request

- Request 객체를 활용하여 요청을 하는 주소와 방식에 대해 확인해 보자

```
from fastapi import FastAPI, Request
app = FastAPI()

@app.get("/")
def main(req: Request):
    print(f'method : {req.method}')
    print(f'url : {req.url}')
    return {"info": f"method:{req.method} {req.url}"}
```

Request 정보를 얻기 위한 라이브러리 호출

Request 객체로 부터 얻는 정보의 일부

실습파일 : 02_request/main.py

Request 와 Parameter

request

- 아래와 같은 요청은 GET 방식에, parameter 가 전송되었다고 한다.

http://127.0.0.1:8000/setInfo?userName=김지훈&gender=male&hobby=독서&hobby=게임&hobby=축구

GET 방식 요청이기에 parameter 가 URL 에 노출되어 있다.

- Request 에서 전달하는 정보와 파라미터를 받아보자

```
@app.get("/setInfo")
def setInfo(req:Request):
    print(f'method : {req.method}')
    print(f'host : {req.client.host}')
    print(f'port:{req.url.port}')
    print(f'url : {req.url.path}')
    print(f'userName : {req.query_params.get("userName")}')
    print(f'gender : {req.query_params.get("gender")}')
    print(f'hobby : {req.query_params.getlist("hobby")}')
    return {"msg":"OK"}
```

실습파일 : 02_request/main.py



```
method : GET
host : 127.0.0.1
port:8000
url : /setInfo
userName : 김지훈
gender : male
hobby : ['독서', '게임', '축구']
```

Request 와 Parameter

parameter

- GET 방식의 parameter 는 URL 에 모두 노출된다.
- 그리고 노출하는 형태는 총 두 가지가 있다.
- 각 방식에 따라 server 에서 받는 방식이 어떻게 다른지 알아보자

• 쿼리 방식

GET http://localhost:8000/items/?skip=10&limit=20

```
@app.get("/items")
def read_name(skip:int, limit:int):
    return {"skip":skip, "limit":limit}
```

실습파일 : 03_param/main.py

• 경로 방식

GET http://localhost:8000/items/id/10

```
@app.get("/items/id/{item_id}")
def read_id(item_id:str):
    return {"item_id":item_id}
```

실습파일 : 03_param/main.py

Request 와 Parameter

parameter

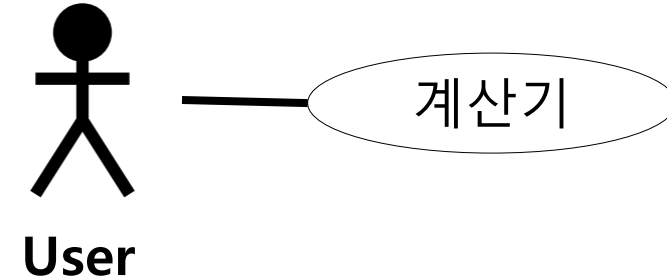
- 3개의 parameter 로 계산을 요청하는 계산기 서비스를 만들어 보자

- view

 /

실습파일 : 04_calc/view/index.html

- Use case Diagram



Summary

request

- Request 객체는 Client 로 부터 온 요청에 관한 모든 정보를 가지고 있다.
- Request 객체로 부터 client 의 요청에 필요한 정보들을 얻어 낼 수 있다.

parameter

- Server 에 보낼 때 추가적으로 보내는 값을 parameter 라고 한다.
- GET 방식과 POST 방식이 있으며 URL 에 파라미터 노출 유무로 구분 할 수 있다.
- 파라미터를 보내는 형태에 따라 쿼리(Query) 방식과 경로(Path) 방식으로 구분 된다.

03.

비동기 통신과 AJAX

비동기 통신과 Ajax

동기와 비동기

- 지금까지 사용해 왔던 방식은 동기방식 이다.
- 동기방식과 비동기 방식의 차이를 먼저 알아보자

항목	동기(Synchronous)	비동기(Asynchronous)
개념	요청한 일이 끝날 때 까지 다른 일을 하지 못함	요청 해 놓고 다른 일을 자유롭게 할 수 있음
예시1	콜센터 전화	예약하면 콜센터에서 전화해 주는 서비스
예시2	줄 서서 식당 대기	내 차례가 되면 전화 해 주는 서비스
예시3	게임 신청 하고 매치 될 때 까지 대기	게임 매치 될 때 까지 다른 기능들을 할 수 있음

비동기 통신과 Ajax

☑ 동기와 비동기

- 앞에서 작성한 04_calc 에 코드를 추가해 동기상황의 특징을 확인해 보자

```
import time
```

```
@app.get('/calc')
```

```
def calc(val1:int, val2:int, oper:str):
```

```
    time.sleep(5) ← 5초 이후 아래 내용을 수행하게 된다.
```

```
    return {"result":result}
```

실습파일 : 04_calc/main.py

- 서버에서 응답이 올 때 까지 기다린다.

3	/	4	전송
--------------------------------	--------------------------------	--------------------------------	-----------------------------------

비동기 통신과 Ajax

동기와 비동기 비교

- 이번엔 동기방식과 비동기 방식으로 서버에 요청해 보자
- Server 는 변경할 내용이 없지만 client 에서는 변경이 필요 하다.

```
<body>
  <h3>동기 방식</h3>
  <form action="/send">
    <input type="text" name="msg" value=""/>
    <input type="submit" value="전송"/>
  </form>
  <a href="axios.html">비동기 방식으로 이동</a>
</body>
```

실습파일 : 05_submit/view/index.html

```
<script>
  function ajaxSend() {
    let val = $('input[name="msg"]').val();
    axios.get('/send?msg='+val).then(function(res){
      console.log(res);
    });
    console.log('전송종료');
  }
</script>
```

실습파일 : 05_submit/view/axios.html

비동기 통신과 Ajax

AJAX

- 우리가 사용하는 axios 라는 라이브러리는 비동기 통신 AJAX 를 지원한다.
- AJAX 는 비 동기로 통신 하는 JavaScript(JSON) 와 xml 이라는 뜻이다.

Asynchronous **J**avaScript **A**nd **X**ml

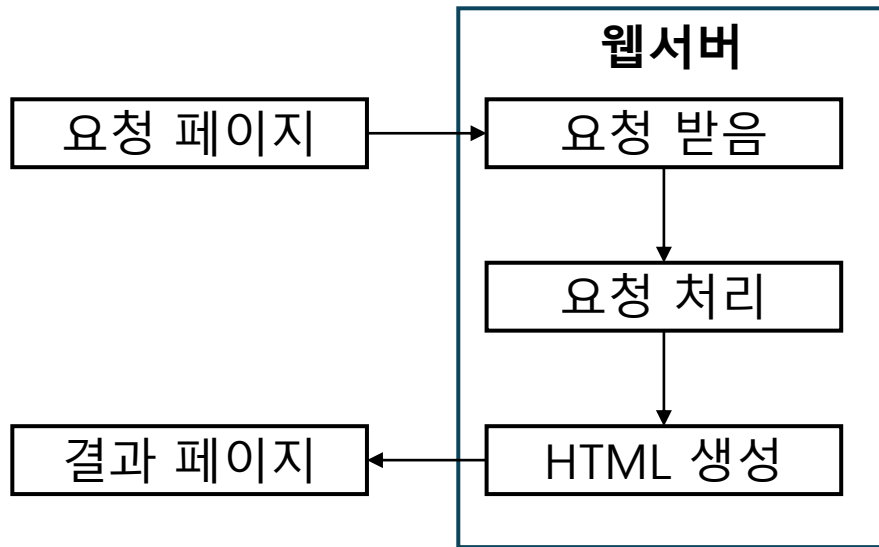
- Java script 를 활용한 AJAX 통신시 XML Http Request 객체를 활용하여 비동기 통신이 가능하다.

비동기 통신과 Ajax

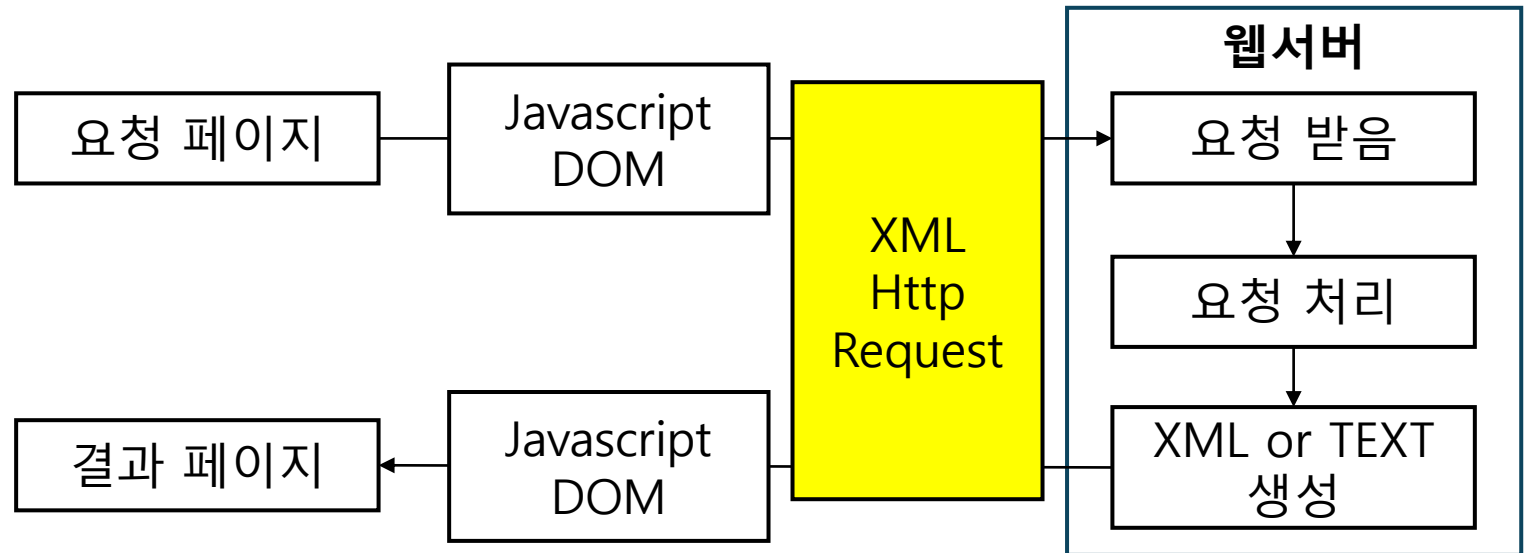
☑ XML Http Request 객체

- 동기와 비동기의 차이점을 생각해 보면 중간에 '**누군가**'가 존재한다는 사실을 알 수 있다.
- **이 존재** 때문에 **요청하는 방식, 결과값을 출력하는 방식** 등이 달라지게 된다.

Form 방식



AJAX 방식

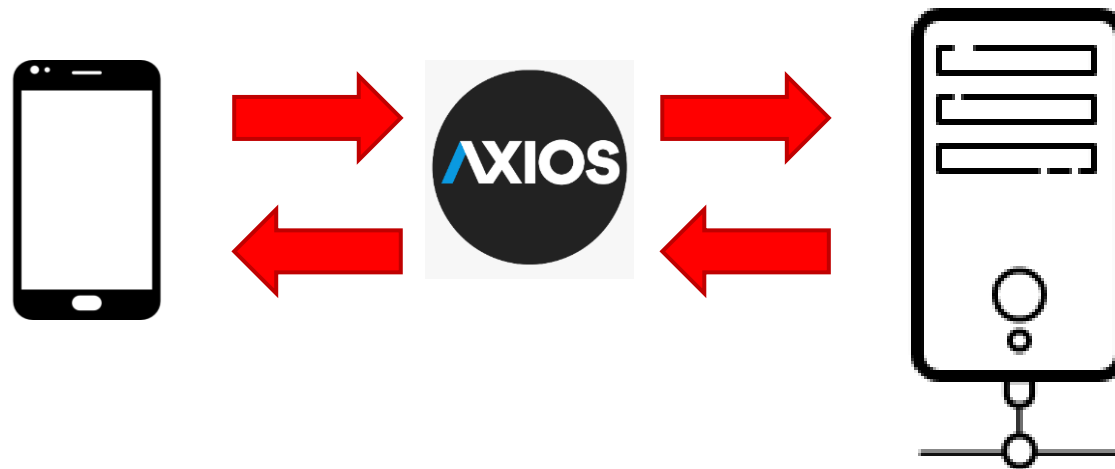


비동기 통신과 Ajax

✓ AXIOS

- XMLHttpRequest 객체를 활용하여 쉽게 비동기 통신을 할 수 있는 라이브러리들이 있다.
- 우리는 가장 사용성이 좋은 axios 를 활용하고 있다.

이름	의존성	특징
fetch()	브라우저 기본 내장 함수	설치가 필요 없으나, 오류처리 및 응답처리가 번거로움
\$.ajax()	J-query 라이브러리 필요	가장 대중적인 라이브러리 이지만 callback 기반임(콜백지옥)
axios()	Axios 라이브러리 필요	사용이 편하고 간결하지만 별도 설치가 필요



Summary

☑ 동기와 비동기 통신

항목	동기(Synchronous)	비동기(Asynchronous)
개념	요청한 일이 끝날 때 까지 다른 일을 하지 못함	요청 해 놓고 다른 일을 자유롭게 할 수 있음
예시1	콜센터 전화	예약하면 콜센터에서 전화해 주는 서비스
예시2	줄 서서 식당 대기	내 차례가 되면 전화 해 주는 서비스
예시3	게임 신청 하고 매치 될 때 까지 대기	게임 매치 될 때 까지 다른 기능들을 할 수 있음

☑ AJAX

- AJAX 는 비 동기로 통신 하는 JavaScript(JSON) 와 xml 이라는 뜻이다.
- XHR(XML Http Request) 객체를 활용하여 응답 값을 비동기로 받아온다.
- fetch() API 나 외부라이브러리(Axios 등) 을 이용할 수 있다.

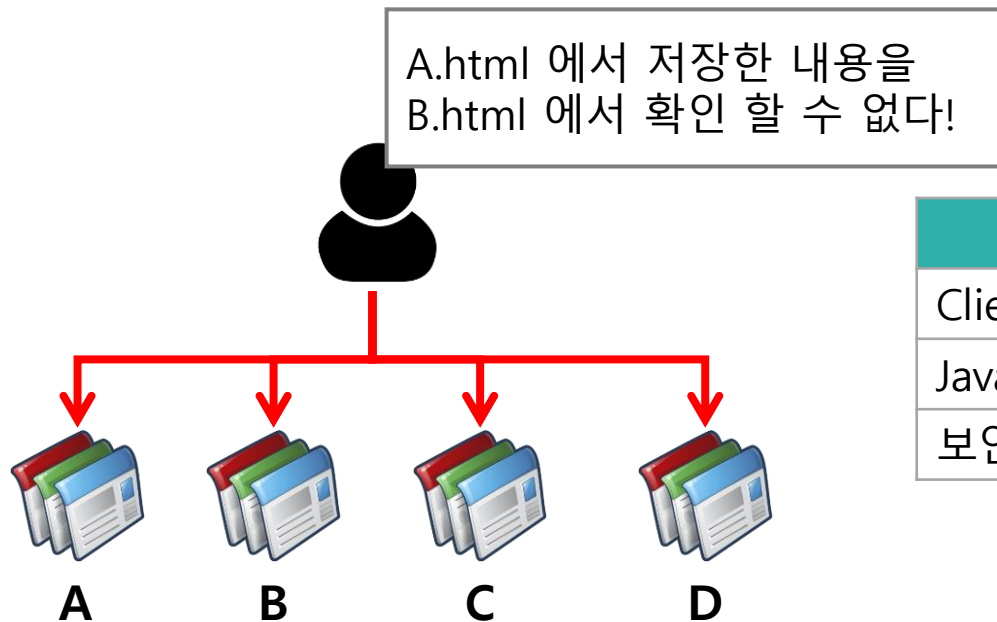
04.

Cookie 와 Session

Cookie 와 Session

☑ 웹 페이지간 저장 장소

- 웹에서 만들어지는 특정한 데이터를 저장 하고 싶을 경우가 있다.
- 이 경우 우리는 cookie 와 session 이라는 저장 객체를 사용 한다.
- 저장 위치가 서로 다르므로 사용법과 보안 수준이 다르다.



Cookie	Session
Client 에 저장(사용자 브라우저)	Server 에 저장
Java script 로도 제어 가능	Server-side program 으로만 제어 가능
보안에 취약	보안성 우수

Cookie 와 Session

Cookie

- Cookie 는 client(사용자) 에 저장된다.
- 그래서 서버에서 받아올 때 Request 객체를 저장할 때 response 객체를 사용 한다.



Cookie 와 Session

☑ Cookie

- Cookie 를 저장하고 호출하는 과정을 확인해 보자

```
@app.post("/login")
def set_cookie(info:Login, resp:Response):
    print(f'param : {info}')
    age = 60 # 수명은 초 단위
    resp.set_cookie(key="loginYN",value="OK",max_age=age)
    resp.set_cookie(key="user_id",value=info.id,max_age=age)
    resp.set_cookie(key="user_pw",value=info.pw,max_age=age)
    return {"msg":"로그인 성공"}
```

Response 객체에 cookie 값을 저장한다.
max_age 속성으로 수명을 지정한다.

```
@app.get("/read_cookie")
def read_cookie(req:Request):
    map = req.cookies
    print(f'cookie : {map}')
    return {'result':map}
```

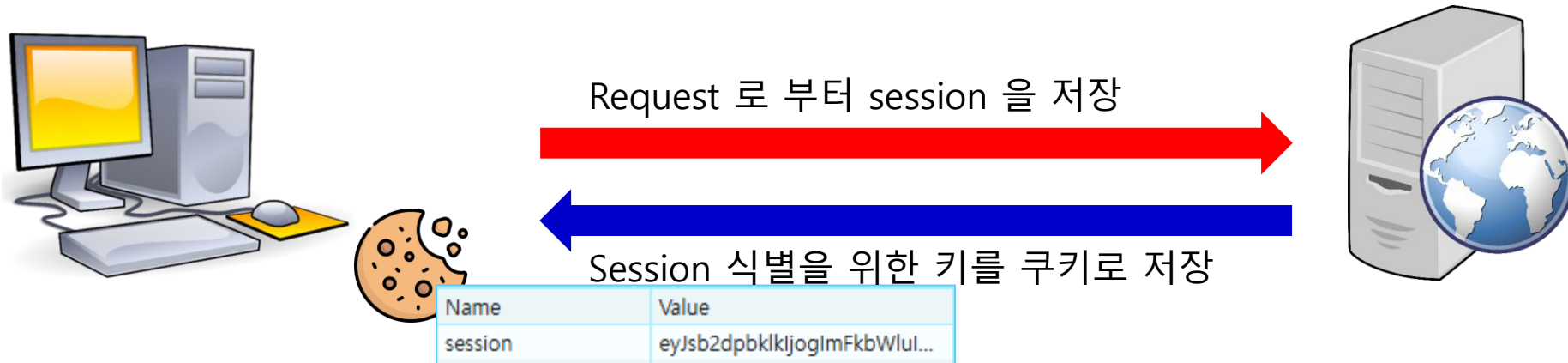
Request 객체로 부터 cookie 를 가져온다.

실습파일 : 06_cookie/main.py

Cookie 와 Session

☑ Session

- Session 은 서버의 메모리에 저장된다.
- 서버에서 session 을 저장하고 session 의 키를 cookie 로 암호화 하여 저장한다.



Cookie 와 Session

Session

- Session 을 저장하고 호출하는 과정을 확인해 보자

```
app.add_middleware(SessionMiddleware, secret_key="session_secret_key")
```

Session 사용시 해야 할 설정

```
@app.get("/session_save")
```

```
def session_save(req:Request):
```

```
    req.session['loginId'] = 'admin'
```

Request 로 부터 session 을 가져와 loginId 라는 속성에 값을 추가

```
    return {"msg":"OK"}
```

```
@app.get("/session_read")
```

```
def session_read(req:Request):
```

```
    loginId = req.session.get('loginId', '')
```

Request 로 부터 session 을 가져와 loginId 속성 값을 가져옴

```
    return {"loginId":loginId}
```

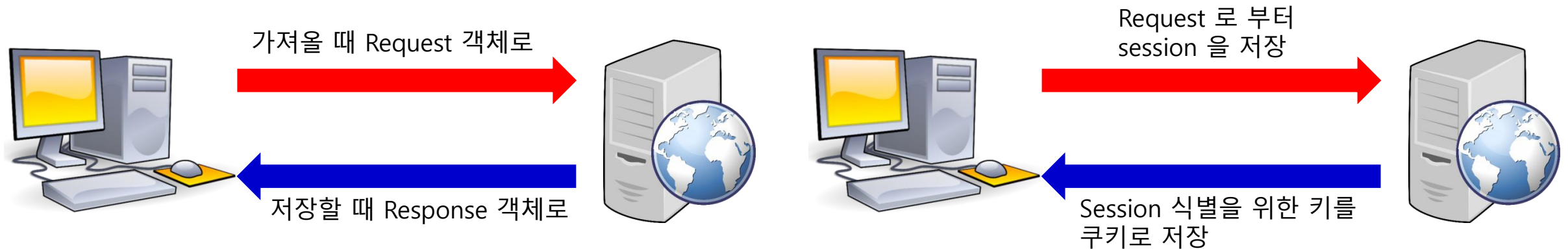
실습파일 : 07_session/main.py

Summary

☑ Session 과 cookie 비교

Cookie	Session
Client 에 저장(사용자 브라우저)	Server 에 저장
Java script 로도 제어 가능	Server-side program 으로만 제어 가능
보안에 취약	보안성 우수

☑ Session 과 cookie 의 저장과 호출



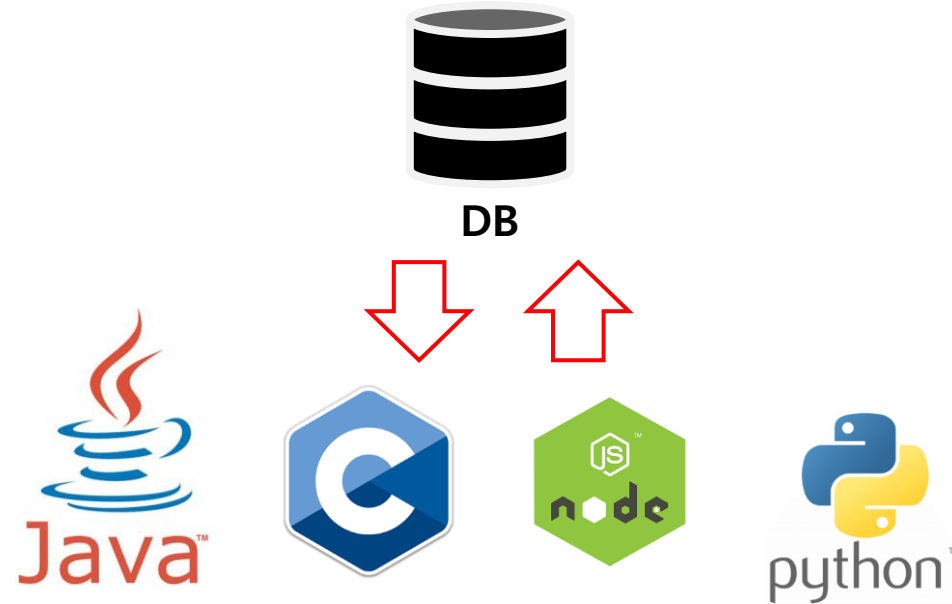
05.

DB 접속과 CRUD

DB 접속과 CRUD

☑ Data Base 의 개념

- Data Base 는 데이터를 **영구적으로** 저장 할 수 있는 저장소
- Data Base 를 이용하면 서로 다른 시스템 간에 데이터를 공유 할 수 있다.



DB 접속과 CRUD

Data Base 와 Python 의 연결

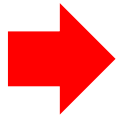
- Python 과 DB 연결, 그리고 쿼리 실행을 위해서는 라이브러리가 추가로 필요하다.

라이브러리	설명	명령어
fast api	Fast API 프레임워크 라이브러리	pip install fastapi
uvicorn	웹서버 라이브러리	pip install uvicorn
pymysql	python 과 DB 를 연결해 주는 라이브러리	pip install pymysql
sqlalchemy	쿼리문을 실행하고 결과값 추출을 도와주는 라이브러리	pip install sqlalchemy

- 이때 requirements.txt 파일을 활용 하면 필요한 라이브러리를 한번에 설치 할 수 있다.

```
fastapi
uvicorn
pymysql
sqlalchemy
```

실습파일 : 08_db_conn/requirements.txt



```
pip install -r requirements.txt
```

DB 접속과 CRUD

Query 문의 실행

- 아래의 함수들을 활용하면 쿼리문을 작성, 실행, 결과 확인이 가능

함수	설명
text(sql)	작성된 쿼리문을 순수 SQL 형태로 인식되도록 저장
execute(sql,param)	text() 에서 작성된 쿼리문을 실행
mappings()	execute() 에서 실행된 결과를 dictionary 형태로 반환 하도록 해 줌
fetchone()	결과값이 하나만 반환할 경우
fetchall()	결과값이 여러 개를 반환할 경우

DB 접속과 CRUD

☑ 회원가입 시스템

- Data Base 를 이용해 회원 정보를 저장해 보자
- 그리고 ID 가 중복되지 않도록 중복 체크 기능도 만들어 보자

Actor

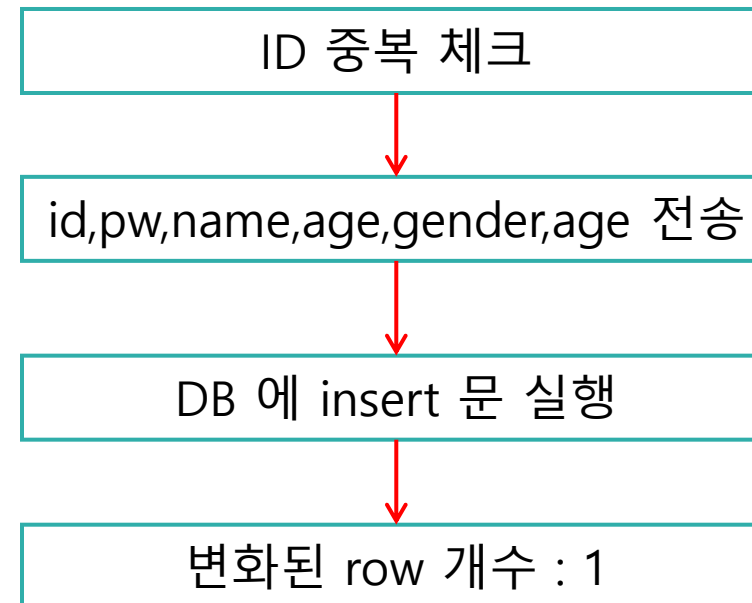
Use Case



User

가입

실습파일 : 09_member_join



DB 접속과 CRUD

✓ 회원가입 시스템

- 회원가입이 정상적으로 진행되고 나면 변화된 row 는 1 이라고 나오지만, 실제 DB 에는 데이터가 들어가 있지 않다

{row: 1}

A-Z id ▼	A-Z pw ▼	A-Z name ▼	123 age ▼	A-Z gender ▼	A-Z email ▼

- 로그를 확인해 보면 ROLLBACK 이 되었음을 알 수 있다.

2025-12-15 16:50:50,339 INFO sqlalchemy.engine.Engine **ROLLBACK**

DB 접속과 CRUD

Commit 과 rollback

- Sqlalchemy 에서는 기본적으로 auto commit 을 off 로 사용한다.
- 작성된 파일에 commit 과 rollback 을 추가해 보자

try:

```
result = db.execute(sql, info)
```

```
row = result.rowcount
```

✓ db.commit() ← 정상적으로 실행되었으면 commit 으로 확정

except Exception as e:

↩ db.rollback() ← 예외 발생시 실행 쿼리를 roll back 으로 되돌림

finally:

db.close() ← 성공/실패 여부와 관계없이 항상 세션을 닫음

```
return {'row': row}
```

실습파일 : 09_member_join/main.py

DB 접속과 CRUD

logger

- Sql 에 관련된 로그를 보면 출력시간과 레벨이 표출된다.
- 이러한 요소들이 이후 로그 내용을 확인하는데 있어 많은 도움을 준다.

```
2025-12-15 17:05:34,752 INFO sqlalchemy.engine.Engine COMMIT
```

표 현	설 명
%(name)s	로거의 이름 (예: __name__으로 지정한 경우 모듈 이름)
%(levelname)s	로그 레벨 (DEBUG > INFO > WARNING > ERROR > CRITICAL)
%(asctime)s	로그가 기록된 시간. 기본 형식은 YYYY-MM-DD HH:MM:SS,sss
%(pathname)s	로그를 호출한 소스 파일의 전체 경로
%(filename)s	로그를 호출한 소스 파일의 파일 이름
%(lineno)d	로그를 호출한 소스 파일 내의 줄 번호
%(funcName)s	로그 레코드를 생성한 함수 이름
%(message)s	로그 메시지 자체

DB 접속과 CRUD

✓ logger

- (1번) logger 설정을 추가해 보자
- (2번) 그리고 기존 print 구문을 logger 로 변경해 보자

1

```
import logging
logging.basicConfig(
    level=logging.INFO,
    format='%(levelname)s:    [%(name)s] %(message)s - %(asctime)s ',
    datefmt='%Y-%m-%d %H:%M:%S',
)
logger = logging.getLogger(__name__)
```

```
@app.get("/overlay")
def overlay(id: str):
    #print(f'id={id}')
    logger.info(f'id={id}')
```

실습파일 : 09_member_join/main.py

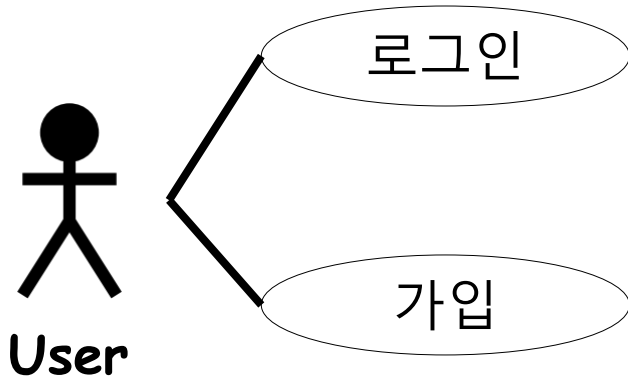
DB 접속과 CRUD

☑ 회원가입과 로그인

- 그동안 학습한 내용을 바탕으로 회원가입과 로그인 기능을 만들어 보자

Actor

Use Case

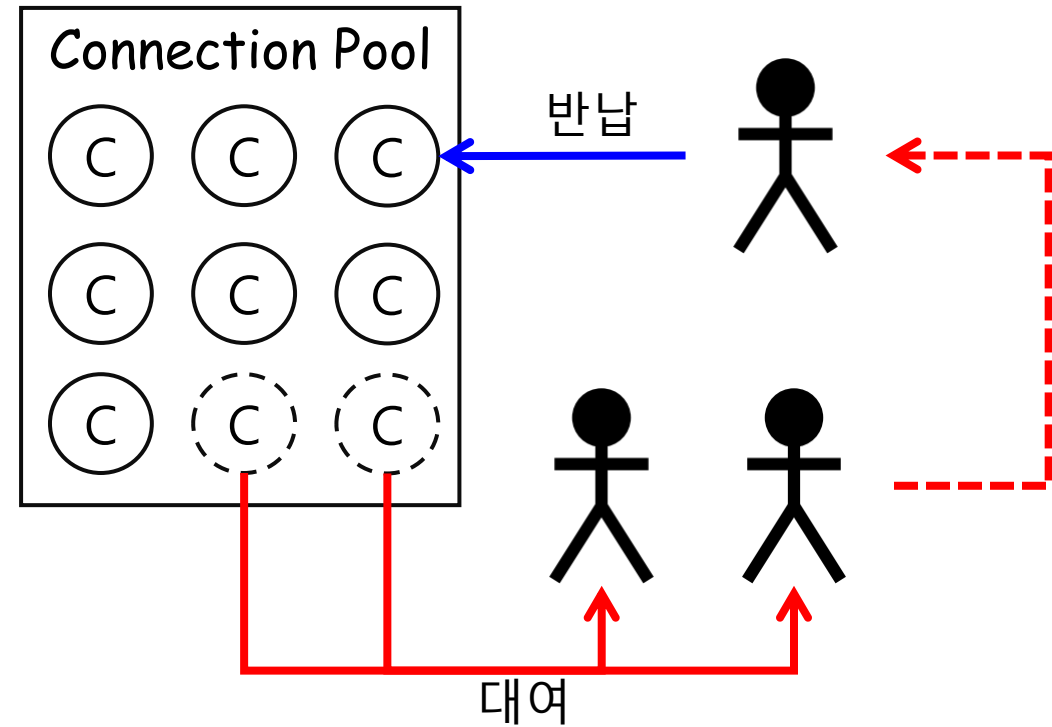
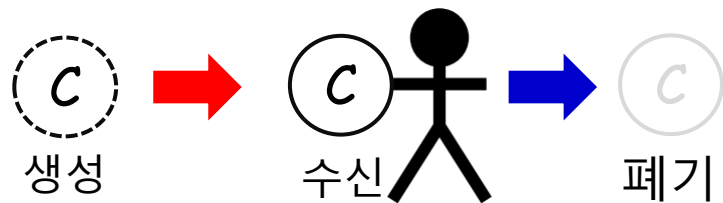


실습파일 : 10_member

DB 접속과 CRUD

✓ Connection pool

- DB 에 접근 하기 위해서는 connection 객체가 필요 하다.
- 이런 객체를 하나씩 만들어 내는데 한계가 있다.
- 그래서 pool 이라는 개념이 탄생하게 된다.



DB 접속과 CRUD

Connection pool

- Connection pool 설정은 기존 create_engine 함수 실행 시 추가 할 수 있다.

```
engine = create_engine(  
    url,  
    echo=True,           #echo 쿼리 로그 출력 여부  
    pool_size=1,         # 유지할 최대 커넥션 수  
    max_overflow=10,     # 초과 요청시 만들 임시 커넥션  
    수  
    pool_timeout=30,     # 커넥션 최대 대기 시간(초)  
    pool_recycle=3600    # 커넥션 재사용 시간(초)
```

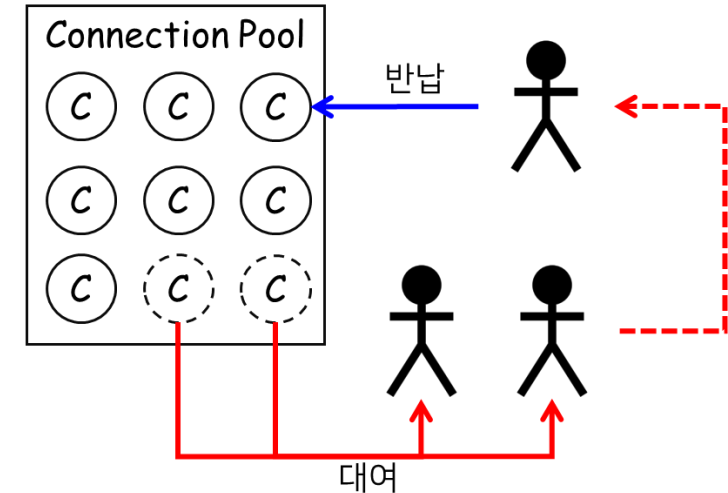
실습파일 : 10_member/database.py

Summary

Database 접속 및 실행

- Python과 mariadb 연결을 위해서는 pymysql 과 sqlalchemy 라이브러리가 필요
- 아래 함수를 통해 쿼리문을 실행 할 수 있다.
- Connection Pool 개념을 통해 connection 을 효율적으로 활용 할 수 있다.

함수	설명
text(sql)	작성된 쿼리문을 순수 SQL 형태로 인식되도록 저장
execute(sql,param)	text() 에서 작성된 쿼리문을 실행
mappings()	execute() 에서 실행된 결과를 dictionary 형태로 반환 해 줌
fetchone()	결과값이 하나만 반환할 경우
fetchall()	결과값이 여러 개를 반환할 경우



06.

게시판의 개념과 제작

게시판의 개념과 제작

☑ 게시판을 이용한 서비스들

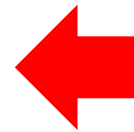
- 게시판은 Data Base 의 CRUD 기능을 모두 사용 할 수 있다.
- Web Service 의 기초 기능은 게시판 기능 이다.
- 많은 Web/app Service 가 게시판을 변형하여 사용 중이다.



coupang
Color Your Days



facebook



모두 DB 의 CRUD 기능을 활용

게시판의 개념과 제작

게시판 제작

- Insert, select, update, delete 문을 활용한 간단한 게시판 서비스를 만들어 보자

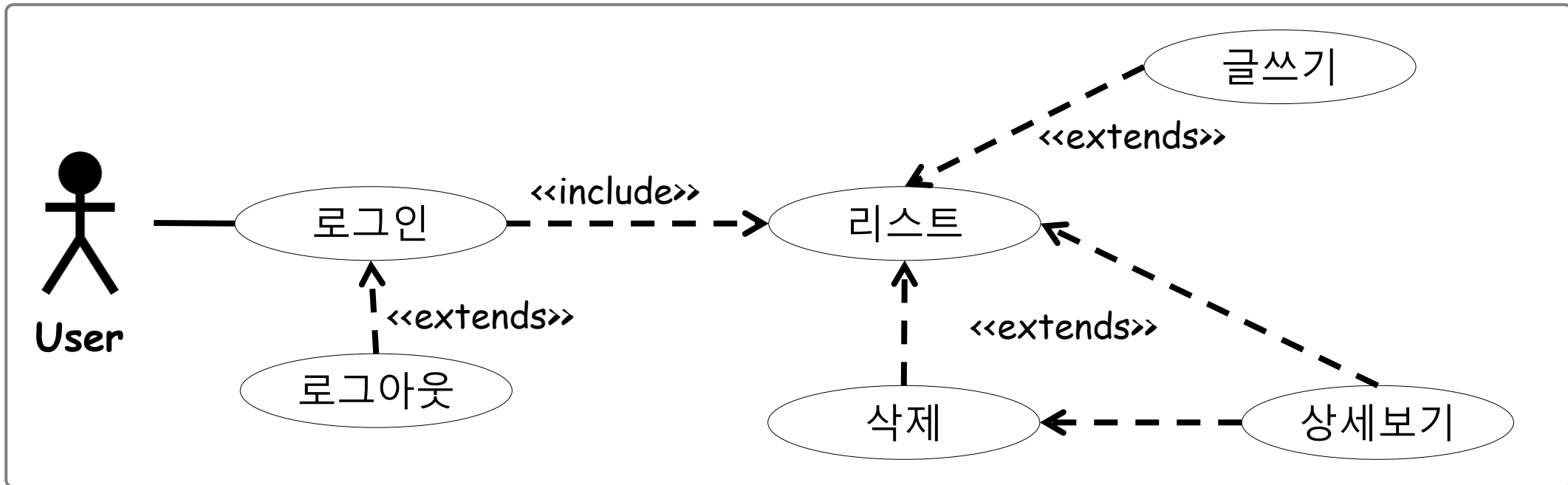
기능	구문	상세 구문
글쓰기	INSERT	INSERT INTO bbs(subject,user_name,content) VALUES(:subject,:user_name,:content)
리스트보기	SELECT	SELECT idx,subject,user_name,reg_date,b_hit FROM bbs ORDER BY idx DESC
상세보기	SELECT	SELECT * FROM bbs WHERE idx=:idx
삭제하기	DELETE	DELETE FROM bbs WHERE idx=:idx

실습파일 : 11_board

게시판의 개념과 제작

✓ 게시판 제작

- 이제 로그인 기능을 추가해 보자
- 게시판 모든 기능은 로그인 없이 사용 할 수 없다.



실습파일 : 12_total

게시판의 개념과 제작

☑ 페이지 처리

- 우리가 리스트를 작성 하다 보면 그 개수가 많아 지게 된다.
- 이 경우 우리는 특정 개수만큼 페이지를 나누어 보여주게 된다.
- 이때 limit 와 offset 을 활용하면 된다.

1
2
3
4
5
6
7
8
9
10

```
SELECT * FROM bbs
```



1	2	3	4
---	---	---	---

```
SELECT * FROM bbs limit 4
```

5	6	7	8
---	---	---	---

```
SELECT * FROM bbs limit 4 offset  
4
```

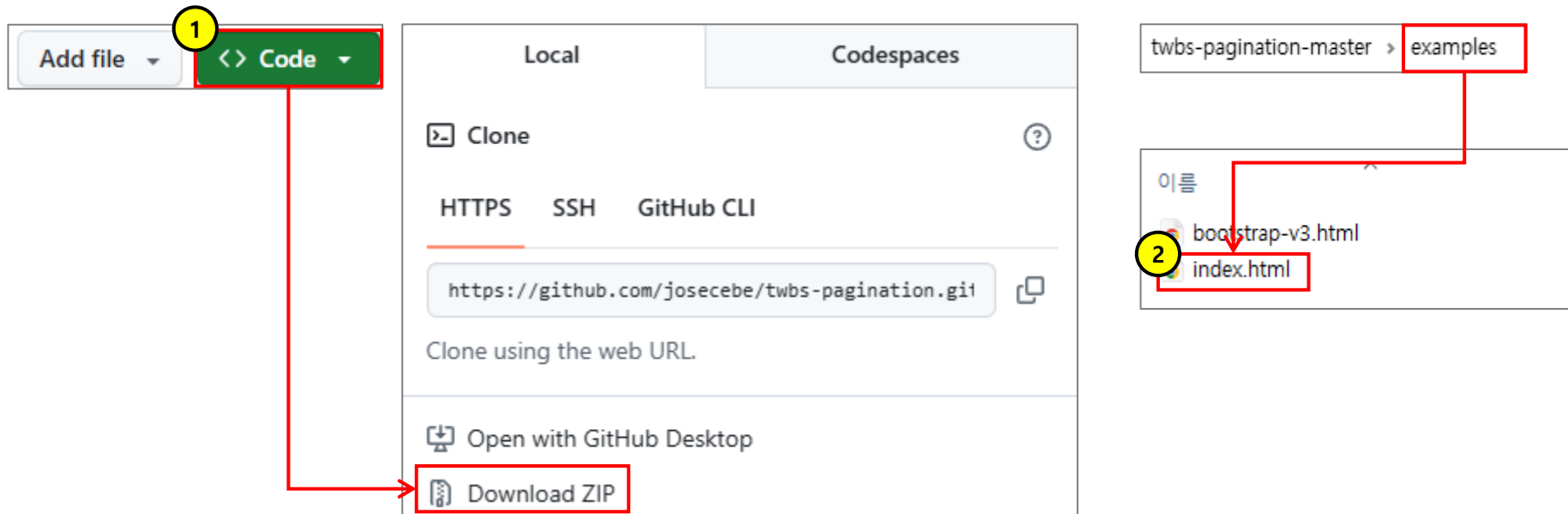
9	10
---	----

```
SELECT * FROM bbs limit 4 offset  
8
```

게시판의 개념과 제작

☑ 페이지 처리

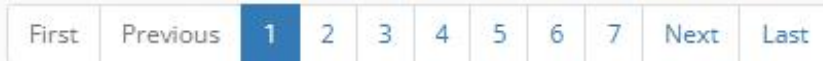
- 페이지 처리에 대한 UI 는 UI 라이브러리를 활용 할 수 있다.
- (1번) <https://github.com/josecebe/twbs-pagination> 에서 파일 다운로드 후 압축 해제
- (2번) example > index.html 파일을 실행/확인 해 보면 실행 모습과 적용 소스를 확인 할 수 있다.



게시판의 개념과 제작

☑ 페이지 처리

- 페이지 처리 UI 라이브러리를 적용해 보자
- 그리고 list 의 쿼리도 수정해 주자



Here is corresponding piece of code:

```
$('#pagination-demo').twbsPagination({  
  totalPages: 35,  
  visiblePages: 7,  
  onPageClick: function (event, page) {  
    $('#page-content').text('Page ' + page);  
  }  
});
```

만들 수 있는 총 페이지 수와, 보여줄 페이지 수를 지정해 줘야 한다.

실습파일 : 13_page

Summary

☑ 게시판의 개념

- 게시판은 Data Base 의 CRUD 기능을 모두 사용 할 수 있다.
- 많은 Web/app Service 가 게시판을 변형하여 사용 중이다.



☑ 페이지 처리

- 한번에 많은 데이터를 가져오기 부담스러울 경우 페이지 처리를 활용 할 수 있다.
- DB 에서는 limit 와 offset 을 활용하고, UI 는 라이브러리를 활용할 수 있다.

1	2	3	4
---	---	---	---

```
SELECT * FROM bbs limit 4
```

5	6	7	8
---	---	---	---

```
SELECT * FROM bbs limit 4 offset
```

4

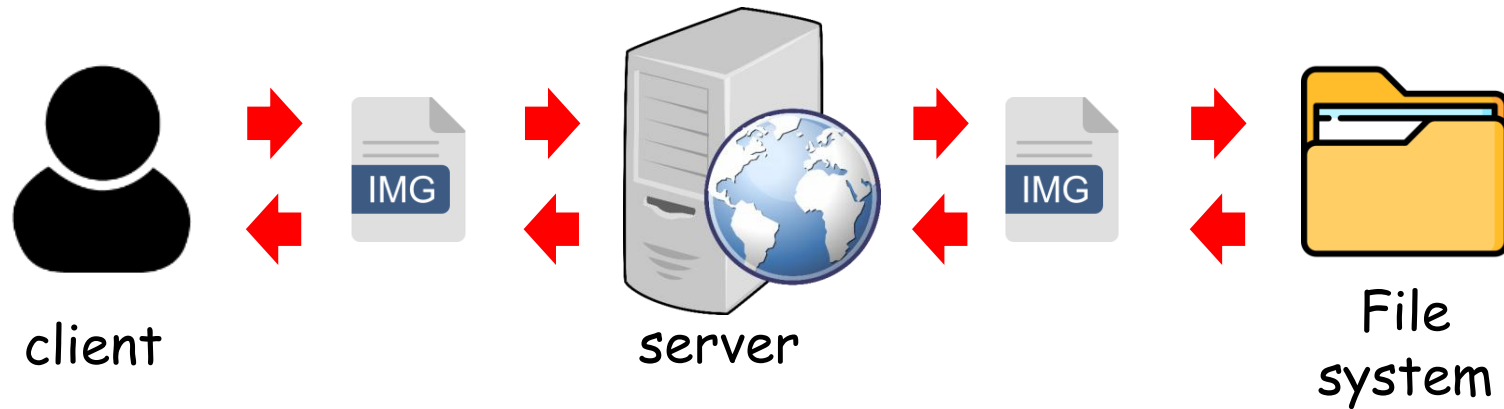
07.

File Upload & Download

File Upload & Download

☑ File 전송

- Server 로의 전송은 text 뿐만 아니라 binary(파일) 도 가능 하다.
- 다만 이를 위해서는 외부 라이브러리가 필요 하다.



File Upload & Download

File 전송

- 다음의 기능을 만들어 보자

기능
파일 업로드
파일 리스트 보기
파일 삭제
파일 다운로드

실습파일 : 14_file

결과 화면(upload.html)

[파일 리스트 보기](#)

파일 업로드

파일 선택 선택된 파일 없음

전송

결과 화면
(fileList.html)
파일 리스트



[삭제](#)

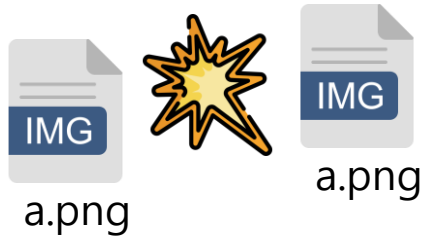


[삭제](#)

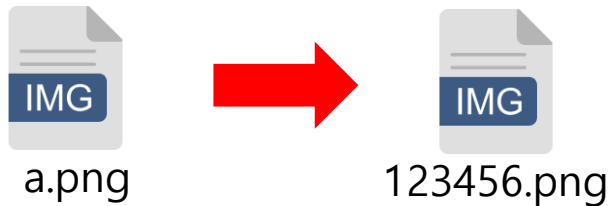
File Upload & Download

File 전송

- 현재 구현한 시스템으로는 같은 파일이 전송될 경우 문제가 발생하게 된다.
- 그래서 기존 파일 이름을 변경하여 저장하도록 변경해 보자



파일명이 같을 경우 충돌이 나거나 over write 되게 된다.



그래서 겹치지 않는 이름으로 변경 후 저장한다.

```
@app.post('/upload')
def upload_file(files: List[UploadFile]):
    logger.info(f'file name : {file.filename}')

    ori_filename = file.filename
    ext = os.path.splitext(ori_filename)[1]
    new_filename = f'{uuid.uuid4()}{ext}'

    path = f'{FILE_PATH}/{new_filename}'
    with open(path, 'wb+') as file_obj:
        shutil.copyfileobj(file.file, file_obj)

    return {"msg": "upload가 완료 되었습니다."}
```

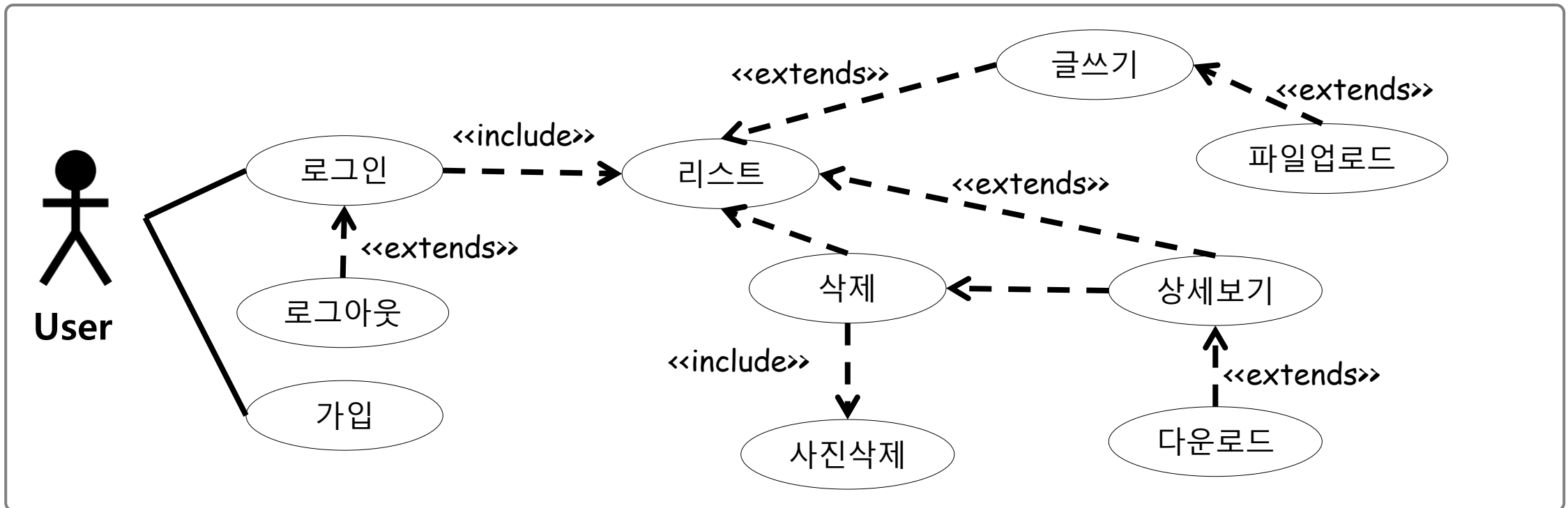
이름 변경 로직 추가

실습파일 : 14_file > main.py

File Upload & Download

✓ 게시판 제작

- 이제는 배운 내용을 토대로 회원 가입, 로그인 부터 사진 첨부 게시판 까지 만들어 보자



실습파일 : 15_file_board

Summary

☑ File Upload & Download

- 파일 전송을 위해서는 post 방식으로 전송을 해야 한다.
- Multi-part 전송을 위한 라이브러리도 필요하다.
- 같은 이름의 파일이 업로드 될 경우 overwrite 방지를 위한 이름변경이 필요



☑ 게시판과의 연동

- 게시판과의 연동을 위해서는 파일관련 테이블이 필요
- 테이블에는 어떤 게시물 번호에 연관 되어있는지, 파일의 원래이름, 변경된 이름 등이 필요
- 파일 등록/삭제 시에도 테이블에서의 등록/삭제가 필요함

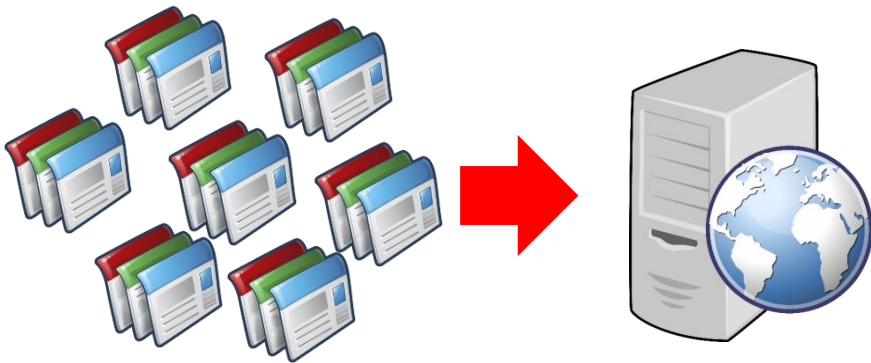
08.

데이터 Crawling

데이터 Crawling

☑ 데이터 수집

- 우리는 데이터를 자체적으로 생산할 수도 있지만 일정 부분은 외부 데이터를 가져와 사용
- Web Crawling 은 프로그램이 Web 을 천천히 돌아다니며(crawling) 자료를 수집해 오는 것



Crawling 의 예시

- | |
|--------------------------------|
| 각 쇼핑몰 의 같은 상품 가격을 가져와 한곳에 모은다. |
| 각 사이트 의 혜택 정보를 가져와 한곳에 모은다. |
| 각 사이트 에 저장된 이미지를 가져와 한곳에 모은다. |

데이터 Crawling

Web Crawling

- Web Crawling 의 라이브러리인 BeautifulSoup 과 Selenium 에 대해서 알아보자

비교항목	Beautiful soup	Selenium
주요 용도	정적 HTML/XML 문서 파싱 (데이터 추출)	웹 브라우저 자동화 및 동적 콘텐츠 처리
동적 페이지	지원 불가 (JavaScript 실행 안 됨)	지원 가능 (AJAX, 스크롤, 클릭 등 처리)
작동 방식	HTML 소스 코드를 텍스트로 읽어 분석	실제 브라우저를 띄워 사람처럼 동작
처리 속도	매우 빠름 (텍스트 기반 처리)	느림 (브라우저 로딩 및 렌더링 대기 시간 발생)
리소스 사용	매우 적음 (가벼움)	높음 (메모리와 CPU 소비가 큼)
설치 및 설정	간단 (라이브러리만 설치)	복잡 (WebDriver 설치 및 버전 관리 필요)

데이터 Crawling

Beautiful Soup

- 속도가 빠르고 CSS selector 를 차용하여 원하는 요소를 추출하기 좋다.
- 프로그램 접근을 막아놓은 사이트에서는 데이터를 가져 올 수 없다.

- Beautiful soup 옵션

BeautifulSoup(markup,parser,options...)

파라미터	종류	내용
markup	html_doc	직접 작성하거나 로컬에서 불러온 HTML 문자열
	response.text	requests나 httpx 등을 이용해 외부 웹 요청의 응답 결과
parser	Html.parser	파이썬 기본 HTML 파서
	lxml	빠른 파서(pip install lxml)
	html5lib	HTML5을 완벽히 지원 하는 파서(pip install html5lib)

데이터 Crawling

Beautiful Soup

- 아래 함수를 이용해 특정 사이트의 내용을 crawling 해 보자

함수	내용
select()	특정 요소를 선택(리스트)
select_one()	특정 요소를 선택(한 개)
elem['attr']	특정 요소의 속성을 다룸
text	특정 요소의 TEXT 를 다룸(html 무시)

```
async with httpx.AsyncClient() as client:
    resp = await client.get(url, params=params)
    soup = BeautifulSoup(resp.text, 'html.parser')
    elements = soup.select('#recruit_info_list div.content div.item_recruit')

    info_list = []
    for elem in elements:
        a_tag = elem.select_one('div.area_job h2.job_tit a')
        title = a_tag.text
        link = a_tag['href']
        date = elem.select_one('div.area_job div.job_date span.date')
        info_list.append({'title': title, 'data': date.text, 'link': link})
```

실습파일 : 16_soup > main.py

데이터 Crawling

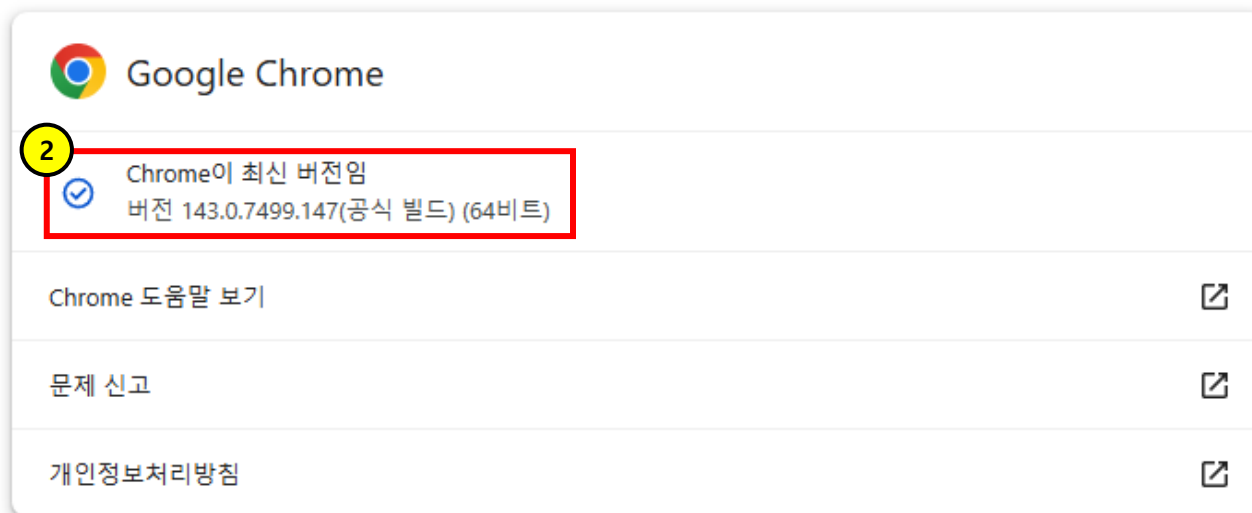
☑ selenium

- 본래는 웹 사이트 테스트 도구(일련의 과정을 자동으로 해 주는 매크로 개념)
- 웹 페이지의 crawler 접근이 막혀 있거나 접근 시 특정한 순서가 있는 경우 유용
- 드라이버 설치를 위해 먼저 chrome 의 버전을 알아보자

- 크롬 우측 상단 메뉴 클릭 후 도움말 > chrome 정보 선택(1번)
- 크롬 버전 확인(2번)



Chrome 정보



데이터 Crawling

selenium

- 해당 버전에 맞는 드라이버를 다운로드 받자
- 아래 링크에서 버전에 맞는 드라이버 버전 링크를 선택 한다.(1 번)
- 링크 : <https://googlechromelabs.github.io/chrome-for-testing/>
- OS 에 맞는 chrome driver 를 다운로드 받자(2 번)

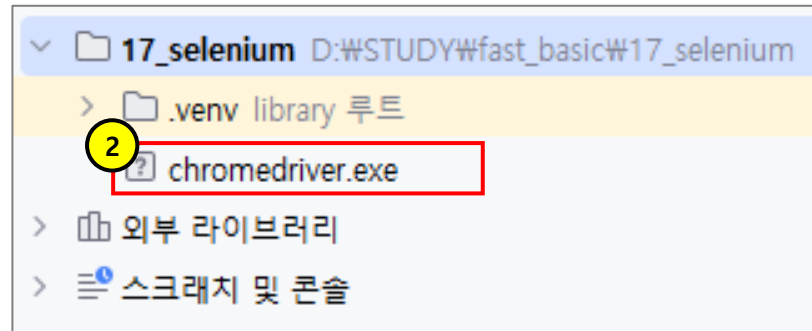
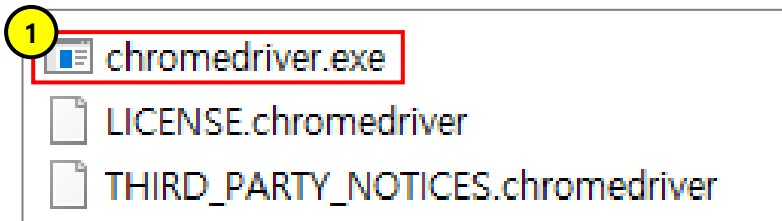
1 <u>Stable</u>	143.0.7499.146	r 1536371	✓
<u>Beta</u>	144.0.7559.31	r 1552494	✓
<u>Dev</u>	145.0.7572.2	r 1556498	✓
<u>Canary</u>	145.0.7583.2	r 1559913	✓

2 chromedriver	win32	https://storage.googleapis.com/chrome-for-testing-public/143.0.7499.146/
chromedriver	win64	https://storage.googleapis.com/chrome-for-testing-public/143.0.7499.146/

데이터 Crawling

☑ selenium

- 다운로드 받은 파일의 압축을 풀어서 프로젝트 안에 넣어주자
 - 압축을 풀어준다(1번)
 - 프로젝트 안에 넣어 준다(2번)



데이터 Crawling

selenium

- Selenium 도 beautiful soup 처럼 특정 요소를 가져올 수 있는 함수와 옵션들을 제공 한다.

함수	설명
elem.find_element()	요소를 하나만 가져 온다.
elem.find_elements()	요소를 List 에 담아서 가져온다.
elem.get_attribute()	요소의 특정 속성을 가져온다.
elem.attribute	

옵션	요소 획득 방법
By.CSS_SELECTOR	Css selector 문법활용
By.Id	id 활용
By.CLASS_NAME	Class 명 활용
By.NAME	name 속성 활용

```
elements = driver.find_elements(By.CSS_SELECTOR, '#recruit_info_list div.content div.item_recruit')
```

```
for elem in elements:
    a_tag = elem.find_element(By.CSS_SELECTOR, 'div.area_job h2.job_tit a')
    title = a_tag.text
    link = a_tag.get_attribute('href')
    date = elem.find_element(By.CSS_SELECTOR, 'div.area_job div.job_date span.date')
    job_list.append({'title': title, 'date': date.text, 'link': link})
```

실습파일 : 17_selenium>main.py

데이터 Crawling

selenium

- Selenium 은 사람이 이용하는 것처럼 특정 이벤트를 발생 시킬 수도 있다.

함수	설명
click()	특정 요소 click
send_keys()	Input type="text" 등에 값 입력
clear()	입력된 값 삭제
submit()	Submit 버튼 click
execute_script()	Java script 문법 실행

이 script 를 실행

```
driver.find_element(By.ID,'id').send_keys(id) ← 특정 요소에 id 값을 입력
driver.execute_script(f'alert("안녕하세요 {id} 님! 사람인 입니다.")')
driver.switch_to.alert.accept() ← 현재 alert 확인버튼 누름
item.click() ← 특정 요소 클릭
```

실행파일 : 17_selenium>main.py

Summary

crawling

- Crawling 은 웹사이트를 돌아다니며 특정 요소를 수집해오는 기능 이다.
- Crawling 을 하는 라이브러리는 beautiful soup 과 selenium 이 대표적이다.
- 각 라이브러리의 장단점이 있으니 적합한 라이브러리를 선택하자

비교항목	Beautiful soup	Selenium
주요 용도	정적 HTML/XML 문서 파싱 (데이터 추출)	웹 브라우저 자동화 및 동적 콘텐츠 처리
동적 페이지	지원 불가 (JavaScript 실행 안 됨)	지원 가능 (AJAX, 스크롤, 클릭 등 처리)
작동 방식	HTML 소스 코드를 텍스트로 읽어 분석	실제 브라우저를 띄워 사람처럼 동작
처리 속도	매우 빠름 (텍스트 기반 처리)	느림 (브라우저 로딩 및 렌더링 대기 시간 발생)
리소스 사용	매우 적음 (가벼움)	높음 (메모리와 CPU 소비가 큼)
설치 및 설정	간단 (라이브러리만 설치)	복잡 (WebDriver 설치 및 버전 관리 필요)

09.

Scheduler 작업

Scheduler 작업

Scheduler

- 어떤 작업들은 주기적으로 실행해야 하는 것들이 있다.
예) 매일 00시 마다 연차 계산, 매월 1일 00시 마다 전월 통계 계산 등
- 이러한 작업들은 사람이 직접 실행 할 수 없기에 스케줄러 라는 기능을 사용 하게 된다.
- Apscheduler 라이브러리를 사용하면 특정 주기마다 일정한 작업을 수행 할 수 있다.

```
pip install apscheduler
```

Scheduler 작업

✓ Scheduler 작업 등록

- 작업할 task 를 만들어 준 다음(1번)
- 스케줄러객체를 생성 한 후 등록해 준다.(2번)

```
def task1(job):  
    print(f'{job} 에 관련된 DB작업 수행')
```

```
def task2(job):  
    print(f'{job} 에 관련된 크롤링 작업 수행')
```

```
def task3(job):  
    print(f'{job} 에 관련된 데이터분석 작업 수행')
```

실습파일 : 18_schedule/job.py

```
sch = AsyncIOScheduler()
```

```
sch.add_job(task1, 'date',  
            run_date='2025-11-13 15:20:00',  
            id='task1', args=['Fast API'])
```

```
sch.add_job(task2, 'interval',  
            seconds=10, id='task2', args=['News 사이트'])
```

```
sch.add_job(task3, 'cron', minute='*/1',  
            day_of_week='MON-FRI', id='task3', args=['수집한 데이터'])
```

실습파일 : 18_schedule/scheduler.py

Scheduler 작업

✓ Job 등록 옵션

```
def add_job(
    self,
    func: Any,
    trigger: str | BaseTrigger = None,
    args: list | tuple = None,
    kwargs: dict = None,
    id: str = None,
    name: str = None,
    misfire_grace_time: int = undefined,
    coalesce: bool = undefined,
    max_instances: int = undefined,
    next_run_time: datetime = undefined,
    jobstore: str = "default",
    executor: str = "default",
    replace_existing: bool = False,
    **trigger_args: Any
) -> Job
```

항목	설명
func	실행함수
trigger	실행주기('cron','interval','date')
**trigger_args	트리거 전달 세부설정(hour=9,minute=0)
args	함수전달 인자값(리스트)
kwargs	함수전달인자값(딕셔너리)
id	작업ID
name	작업이름(설명용)
replace_existing	중복ID 작업의 덮어쓰기여부
jobstore	이 작업을 저장할 Job Store 이름
executor	ThreadPoolExecutor, ProcessPoolExecutor
misfire_grace_time	n 초안에 놓친 작업 실행
coalesce	실행누락이 다수일 경우 마지막것만 실행?
max_instances	동시에 실행 가능한 이 작업의 최대 개수
next_run_time	다음 실행 시간(미지정시 트리거 자동 계산)

Summary

Scheduler

- 주기적으로 실행해야 하는 작업들을 스케줄러 라는 기능을 통해 실행 할 수 있다.
- 우리는 스케줄러 라이브러리 중 APScheduler 를 사용한다.

```
pip install apscheduler
```

Scheduler 등록

- 주기적으로 실행할 Job(task) 을 만들고 Scheduler 객체에 등록하면 된다.
- 사용하고 싶은 주기와 방식에 따라 등록 옵션이 달라진다.

```
sch = AsyncIOScheduler()
sch.add_job(task2, 'interval', seconds=10, id='task2', args=['News 사이트'])
sch.add_job(task3, 'cron', minute='*/1', day_of_week='MON-FRI', id='task3', args=['수집한 데이터'])
```